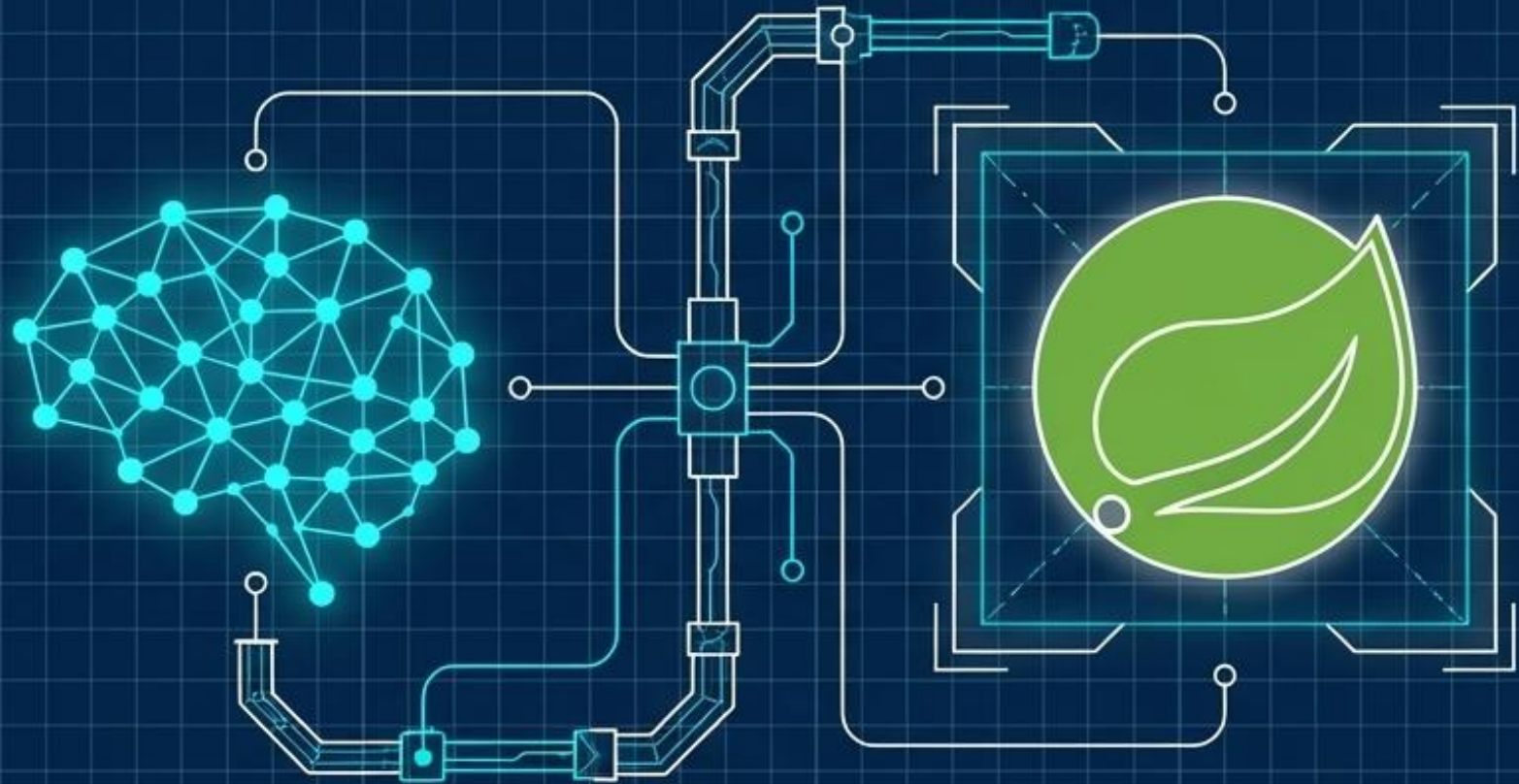


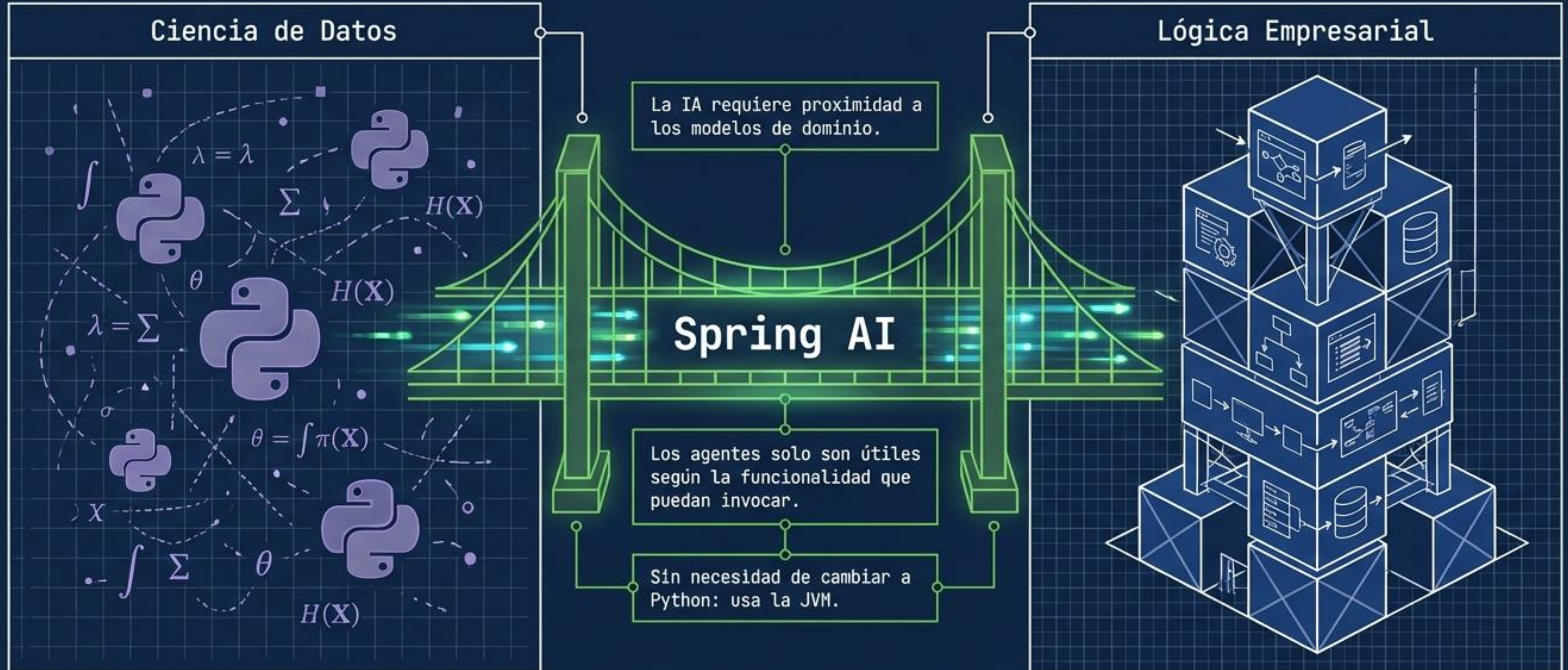
Spring AI en Acción: Llevando la IA Generativa al Ecosistema Java

De prompts básicos a arquitecturas RAG empresariales con rigor de ingeniería.



El Desafío de la IA en la Empresa

El éxito de la IA no se trata solo de modelos; se trata de integración con la lógica de negocio. – Rod Johnson



El Cliente Universal: Abstracción de Modelos

``ChatClient`` aísla tu aplicación de las APIs propietarias.



Portabilidad:

Escribe el código una vez, cambia de modelo sin refactorizar.



Flexibilidad local:

Cambia de un modelo en la nube a modelos locales gratuitos usando Ollama.

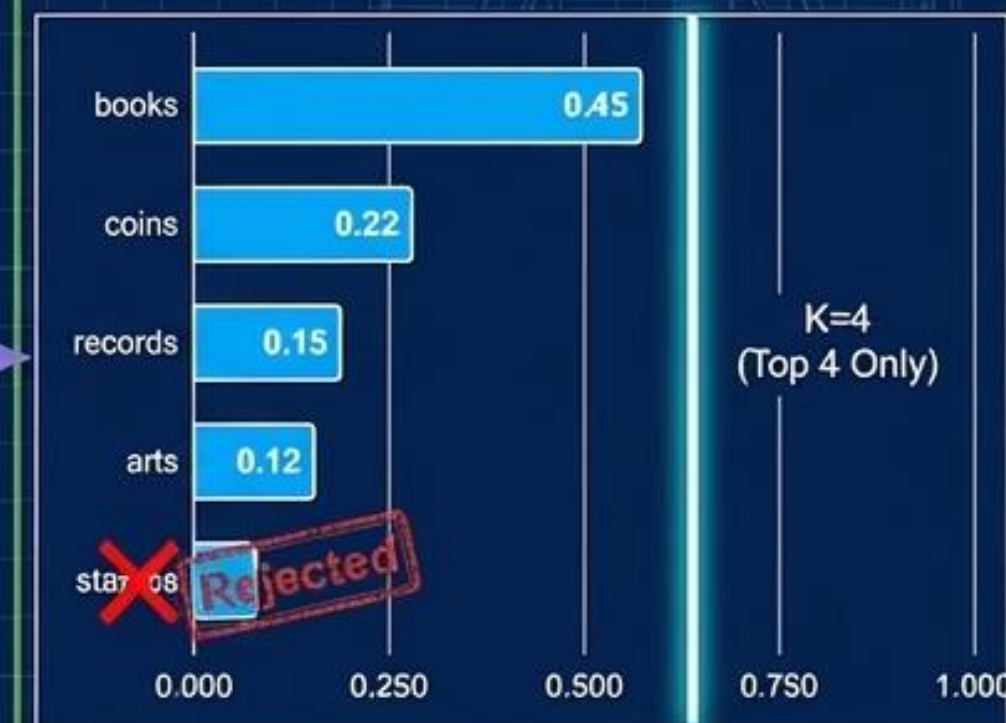
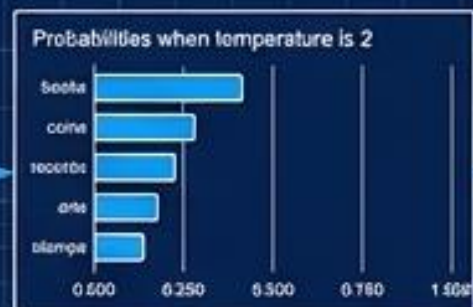
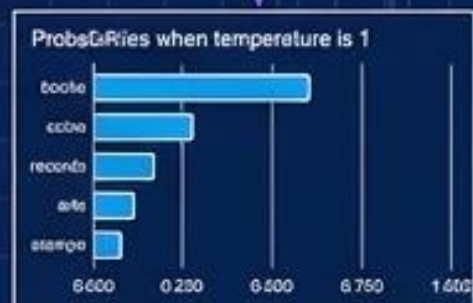
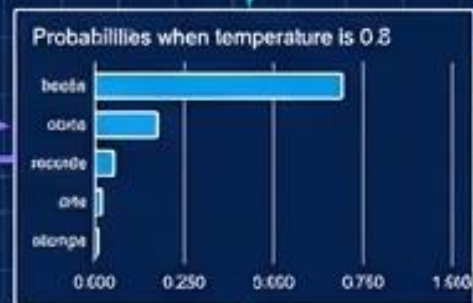
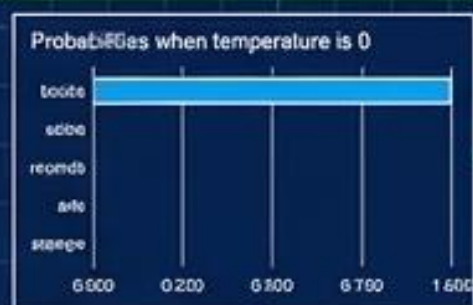


Consistencia:

Interfaz unificada para parámetros, roles y respuestas en cualquier proveedor.

Domando el Nondeterminismo

Parametrizando la creatividad vs. la precisión determinista.



Temperature: Ajusta la aleatoriedad escalando las probabilidades.

Top-P: Recorta la cola acumulativa (Límite dinámico).

Top-K: Límite estricto a las opciones principales (Conteo simple).

Ingeniería de Prompts: De Strings a Plantillas

Reemplazar la concatenación frágil con archivos `StringTemplate`.

Before: Concatenación Frágil

```
String prompt = "Act as a " + role  
               + ". The game is " + gameTitle  
               + ". Answer: "  
               + question;
```

After: Plantillas Seguras con StringTemplate

%% prompt.st:
\n

Act as a {role}.
The game is {gameTitle}.
Answer: {question}.



```
@Value("classpath:prompt.st")  
Resource promptResource;
```

```
// En un método:  
StringTemplate template = StringTemplate.of(promptResource);  
String prompt = template.param("role", role)  
                      .param("gameTitle", gameTitle)  
                      .param("question", question)  
                      .render();
```

Blueprint Notes



Desacoplamiento: Mantiene los prompts separados del código fuente.



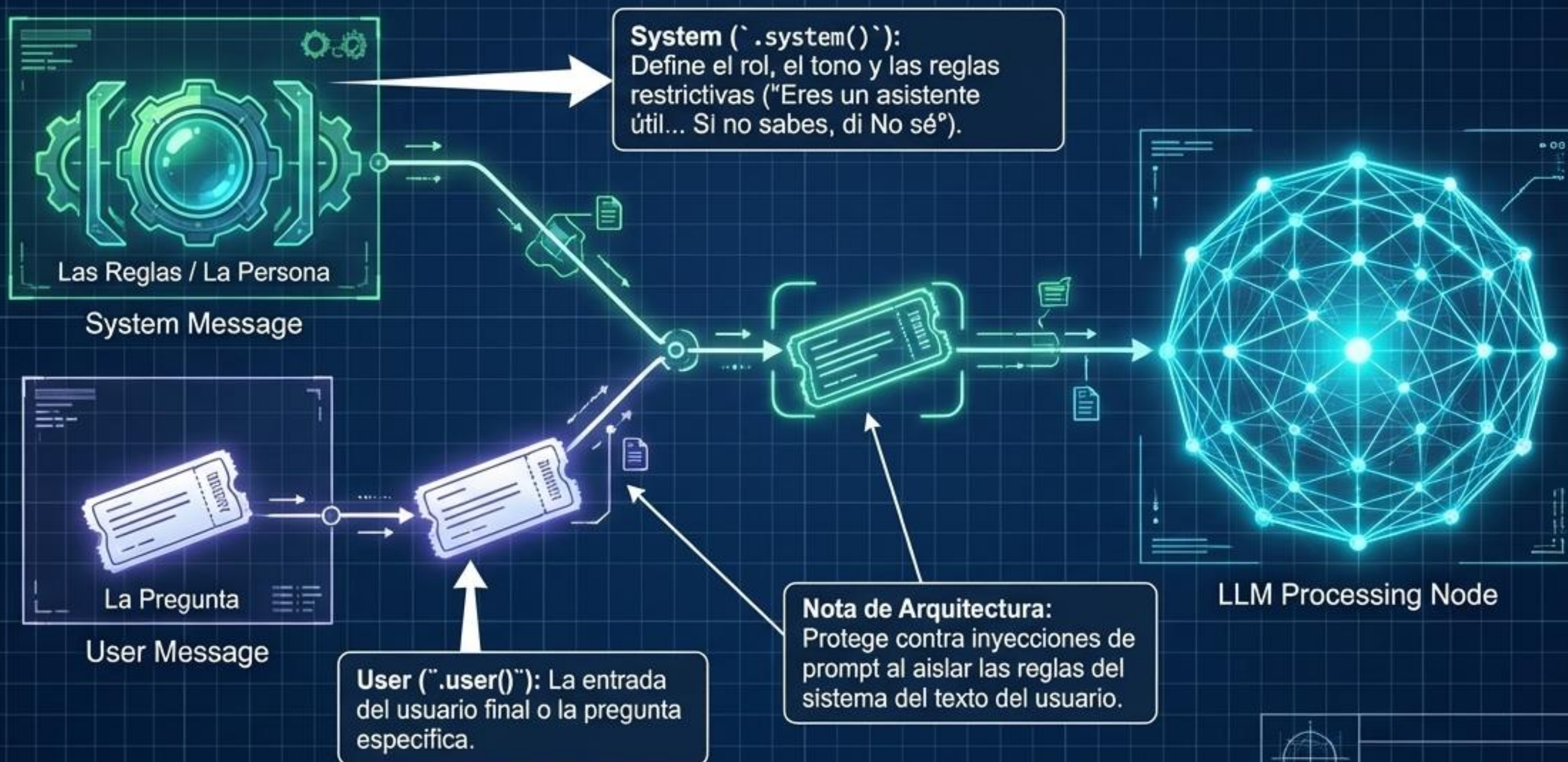
Mantenibilidad: Permite iterar la ingeniería del prompt sin recompilar.



Inyección segura: Usa `param()` para rellenar marcadores como `{gameTitle}` y `{question}` de forma dinámica.

Estructurando la Mente del LLM con Roles

Separar las instrucciones de comportamiento de los datos del usuario.



Conversión de Salida: Respuestas Tipadas

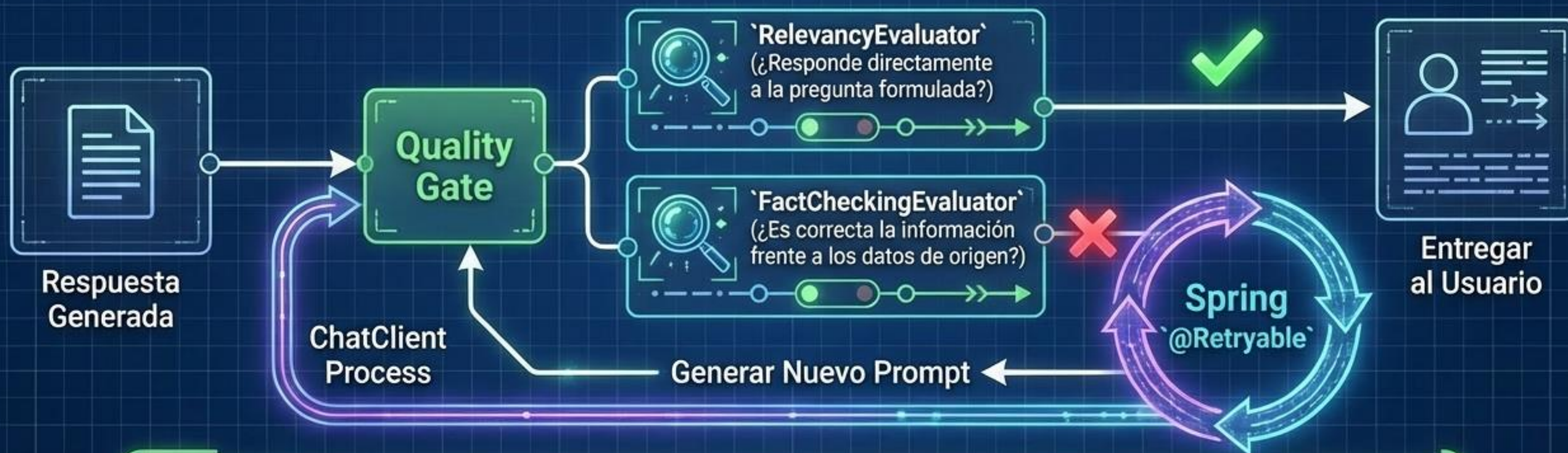
Transformando texto no estructurado en objetos Java robustos.



Inyección automática de esquema JSON (RFC8259) en el prompt y conversión automática del texto generado a un Record o Clase.

Quality Assurance: Evaluando la Generación

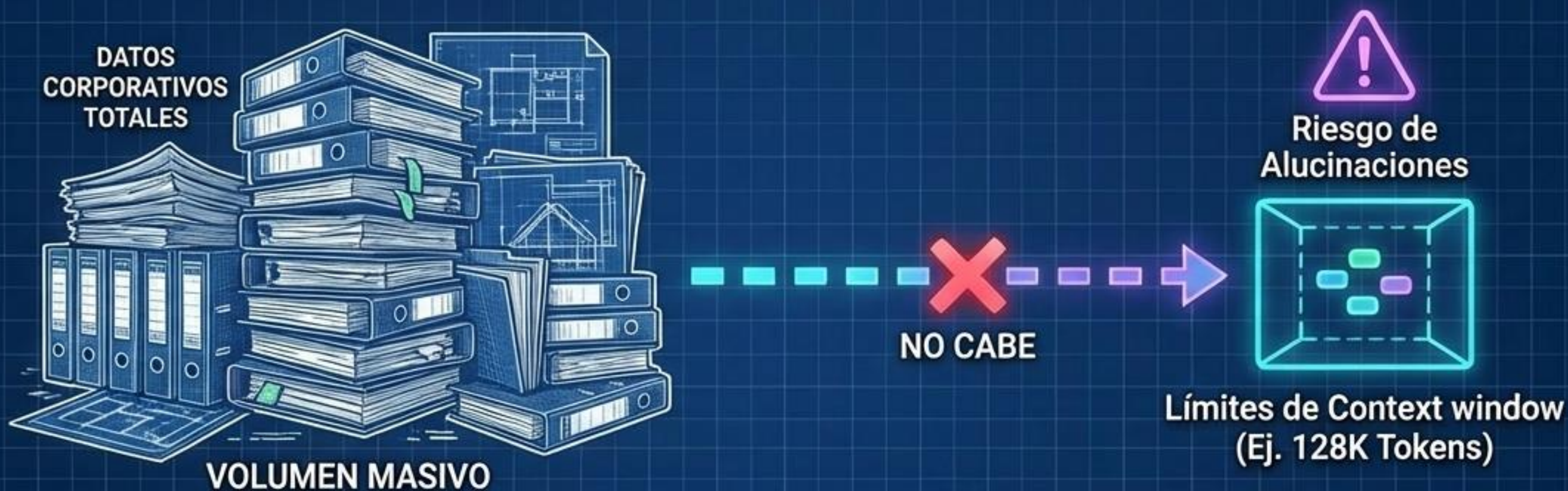
Pruebas automatizadas y validación en tiempo de ejecución para IA probabilística.



Resiliencia empresarial: Uso de Spring Retry para reintentar automáticamente prompts si fallan la evaluación algorítmica.

El Límite del Modelo: Alucinaciones y Contexto

No puedes poner todos los documentos de la empresa en un solo prompt.



Costos

Pagar por tokens de contexto innecesarios.

Límites estrictos

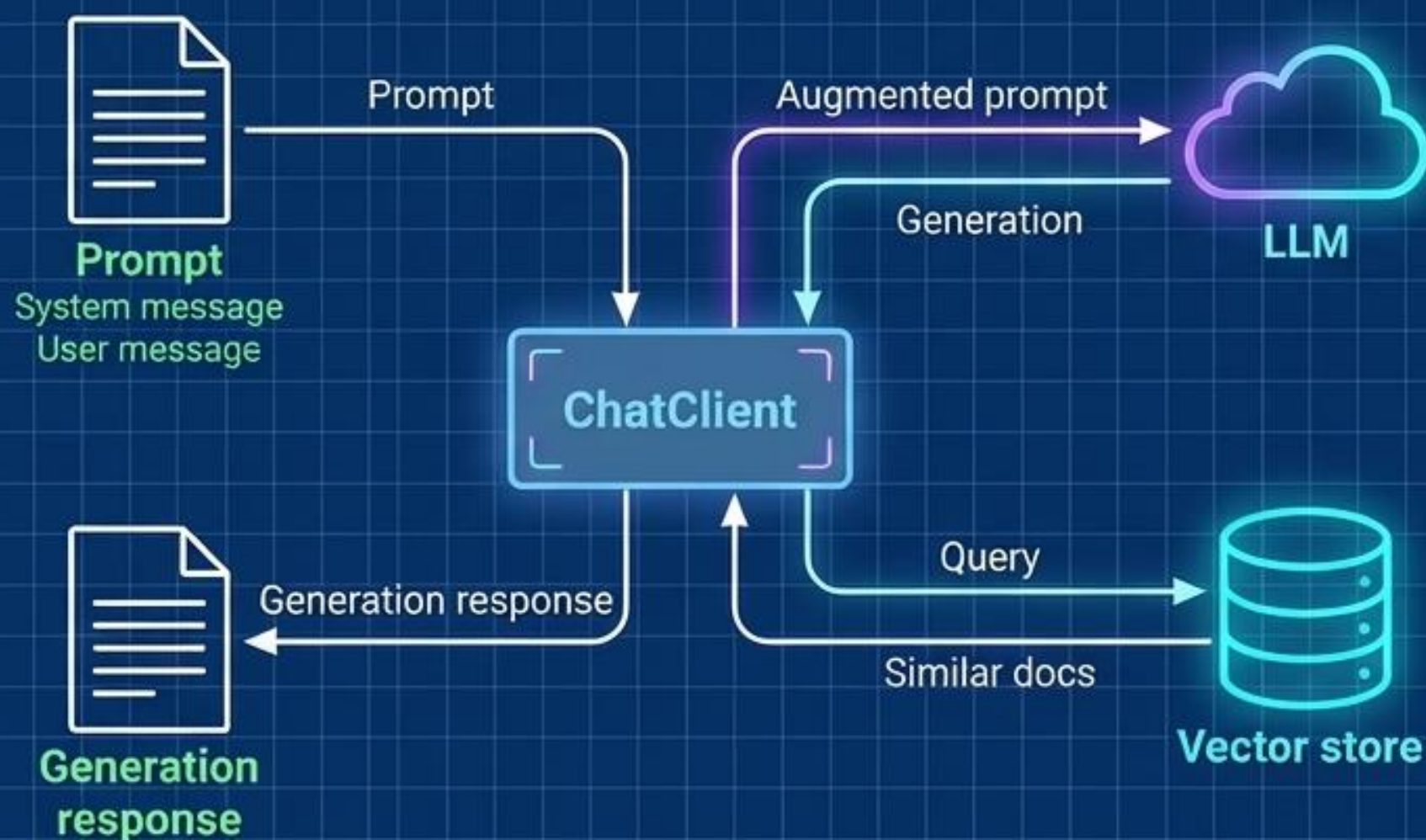
Los modelos tienen ventanas máximas de memoria que no se pueden exceder.

El Peligro

Si un LLM carece de contexto, inventará respuestas falsas en lugar de admitir ignorancia.

La Solución: Generación Aumentada por Recuperación (RAG)

Buscar semánticamente primero, generar respuestas después.



1. Recuperación

Buscar semánticamente en el 'Vector Store' solo los fragmentos de documentos más relevantes para la 'Query'.

2. Aumento

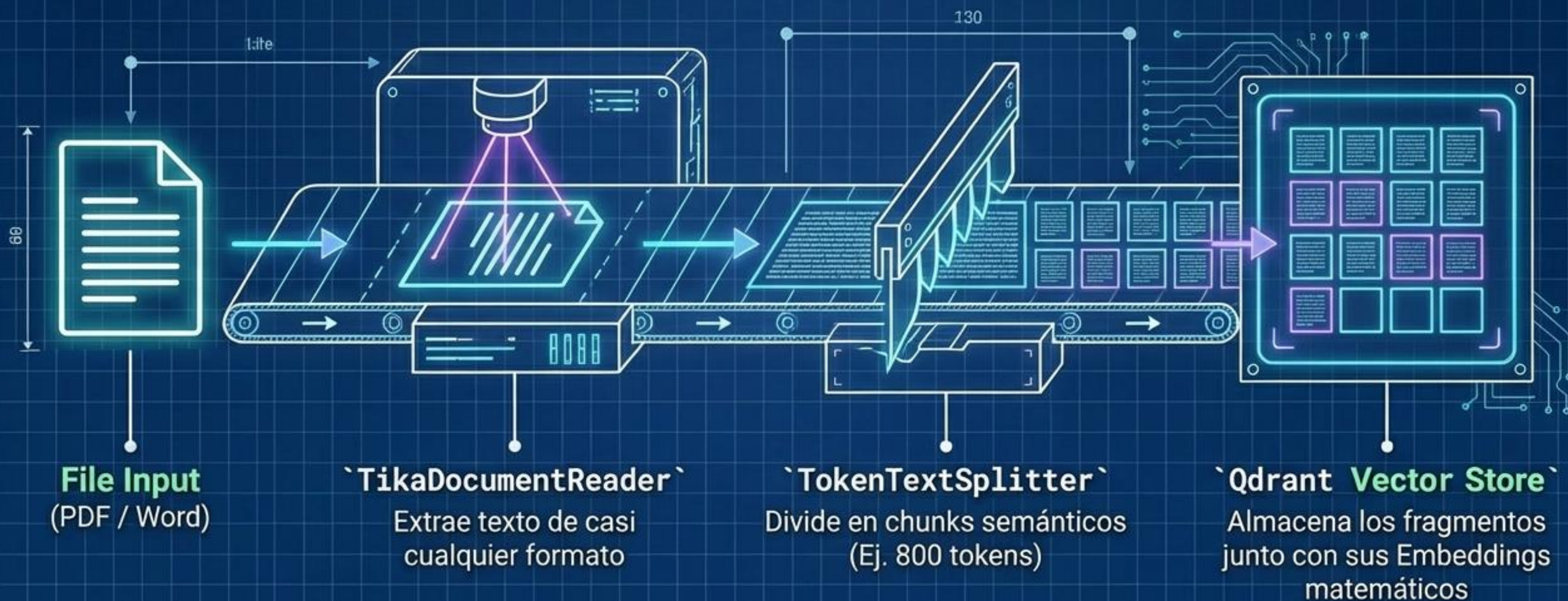
Inyectar los 'Similar docs' recuperados directamente en el 'Augmented prompt' junto al mensaje del sistema.

3. Generación

El LLM responde basándose exclusivamente en los datos inyectados para evitar alucinaciones.

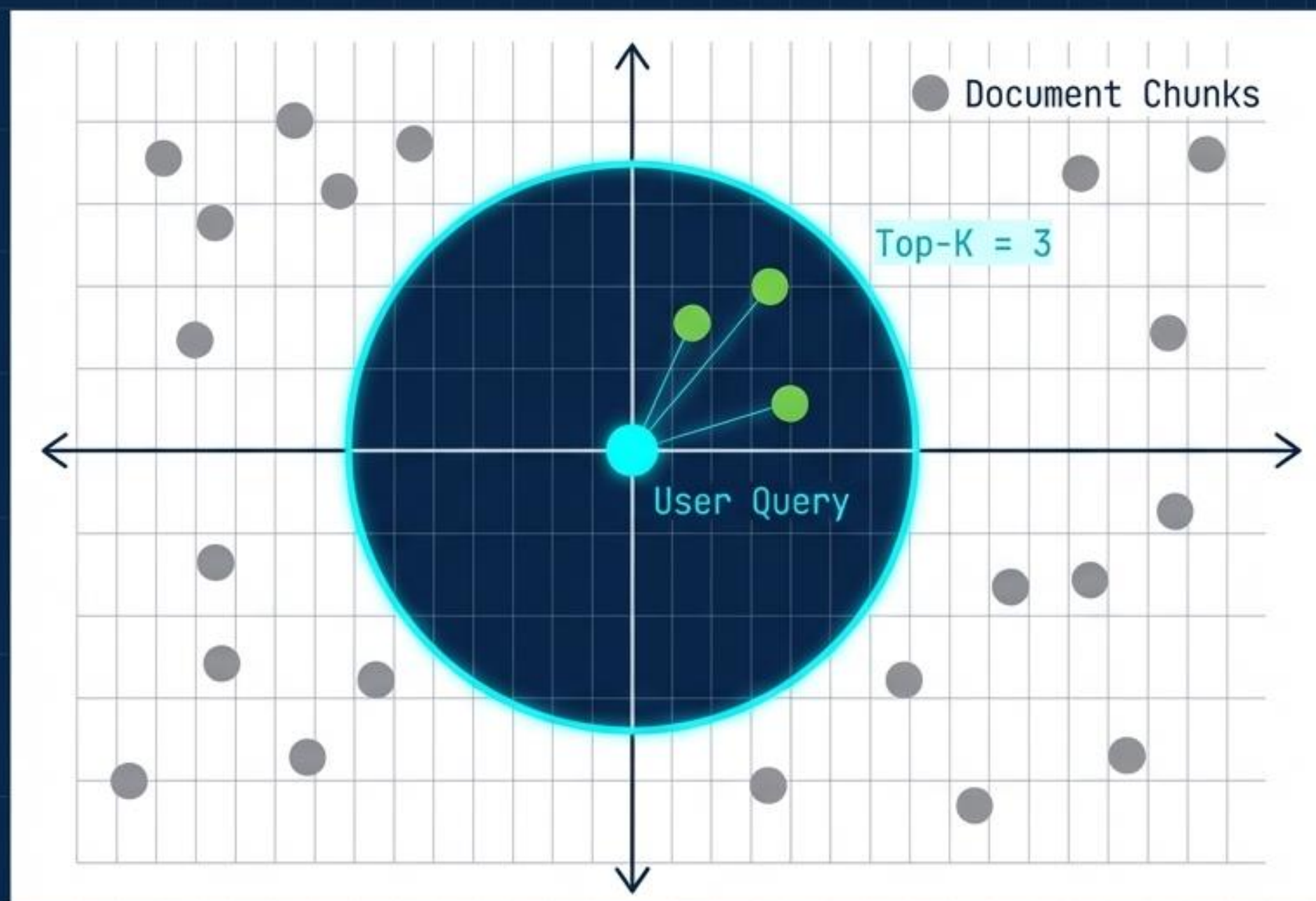
El Pipeline de Ingesta de Datos (ETL para IA)

Preparando documentos no estructurados para búsqueda vectorial.



Búsqueda Semántica: Entendiendo los Embeddings

Encontrar respuestas por significado, no por coincidencia de palabras clave.

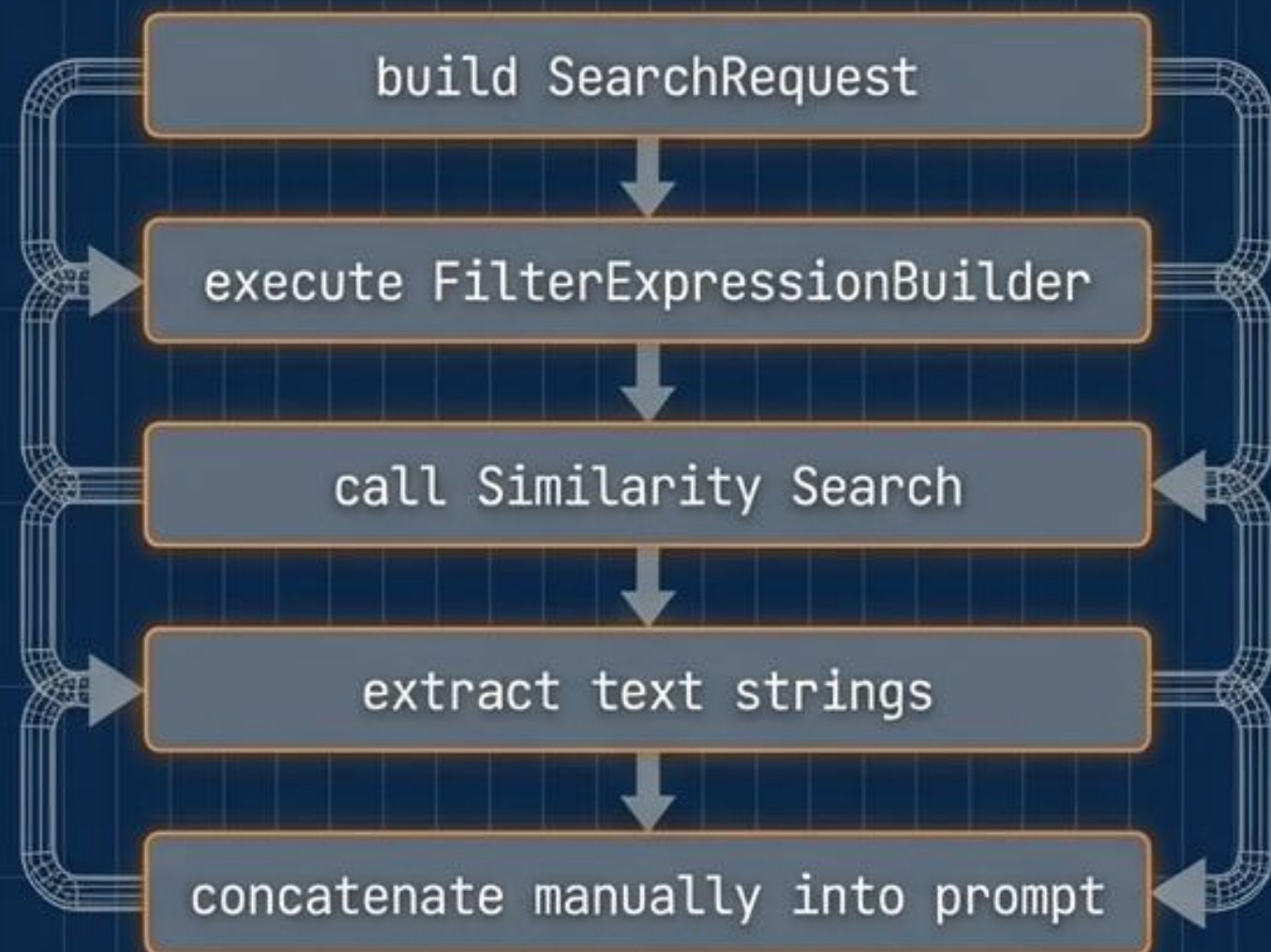


Proceso Matemático	
Embeddings	Embeddings: Modelos de IA traducen texto a coordenadas numéricas.
Similitud del Coseno	Similitud del Coseno: Calcula la distancia angular matemática entre la pregunta y los fragmentos.
Top-K	Retorna solo los fragmentos más cercanos para inyectar en el prompt.

Comparativa RAG: Manual vs. Advisors

Eliminando el código repetitivo con patrones interceptores.

RAG Manual



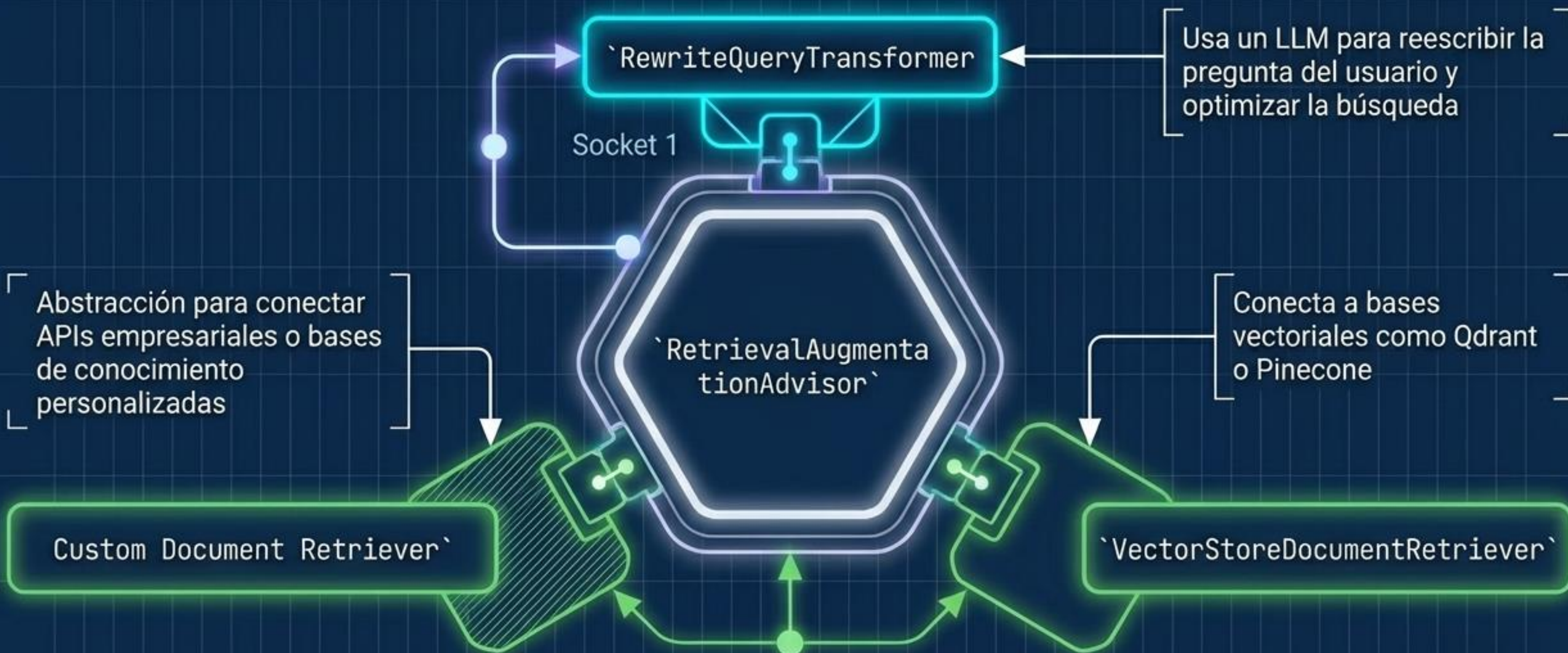
Spring AI Advisor

```
.advisors(QuestionAnswerAdvisor  
    .builder(vectorStore).build())
```

- ✓ Automatiza la búsqueda en la base de datos vectorial.
- ✓ Adjunta automáticamente el contexto recuperado al prompt del sistema.
- ✓ Transforma la implementación de RAG en una única línea de configuración interceptora.

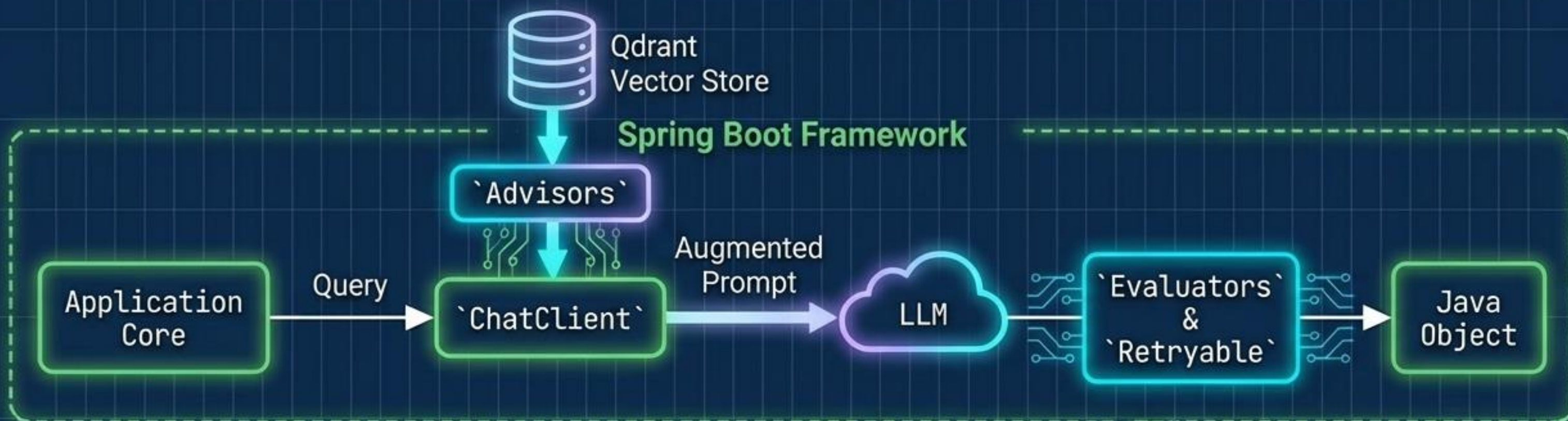
RAG Modular y Expandible

Arquitecturas avanzadas requieren control detallado del flujo de recuperación.



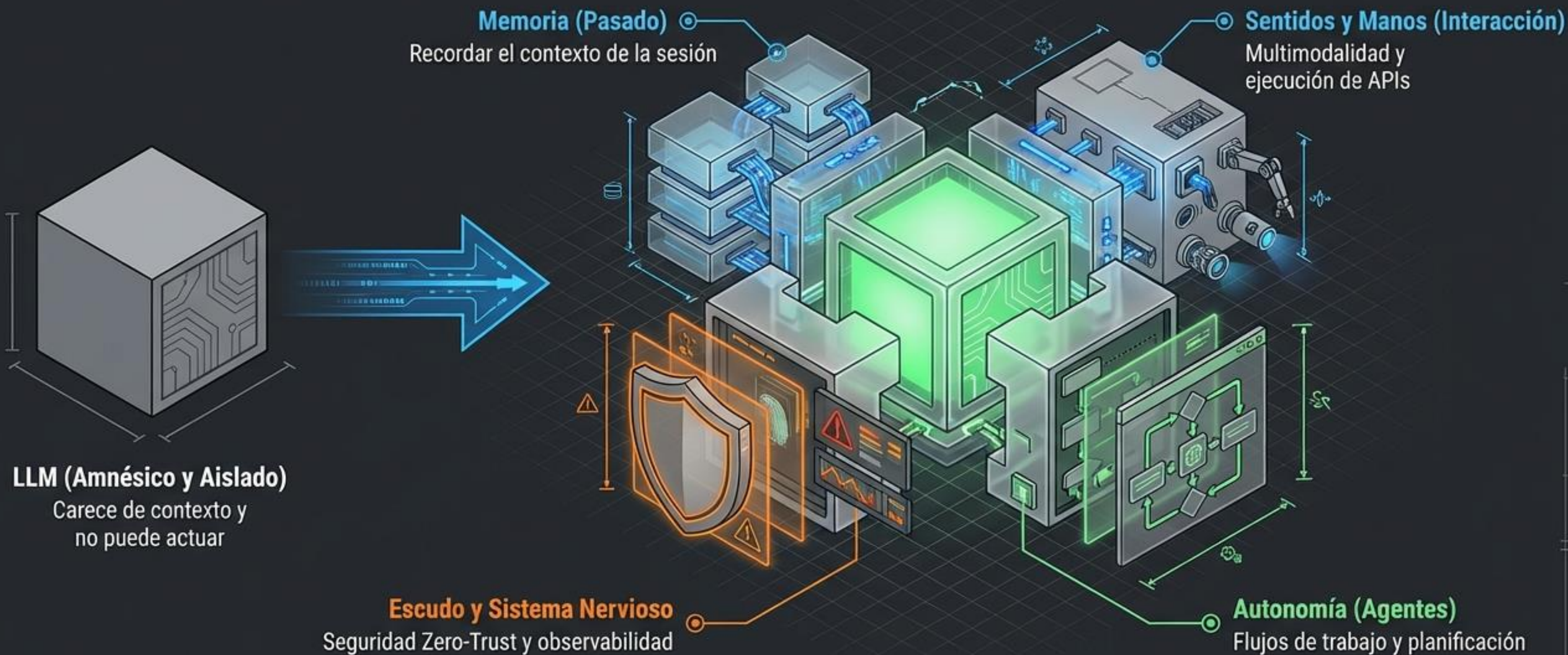
Síntesis: Arquitectura Empresarial de Spring AI

La IA generativa ya no es un experimento aislado de Python; es un componente de software robusto, escalable y tipado.



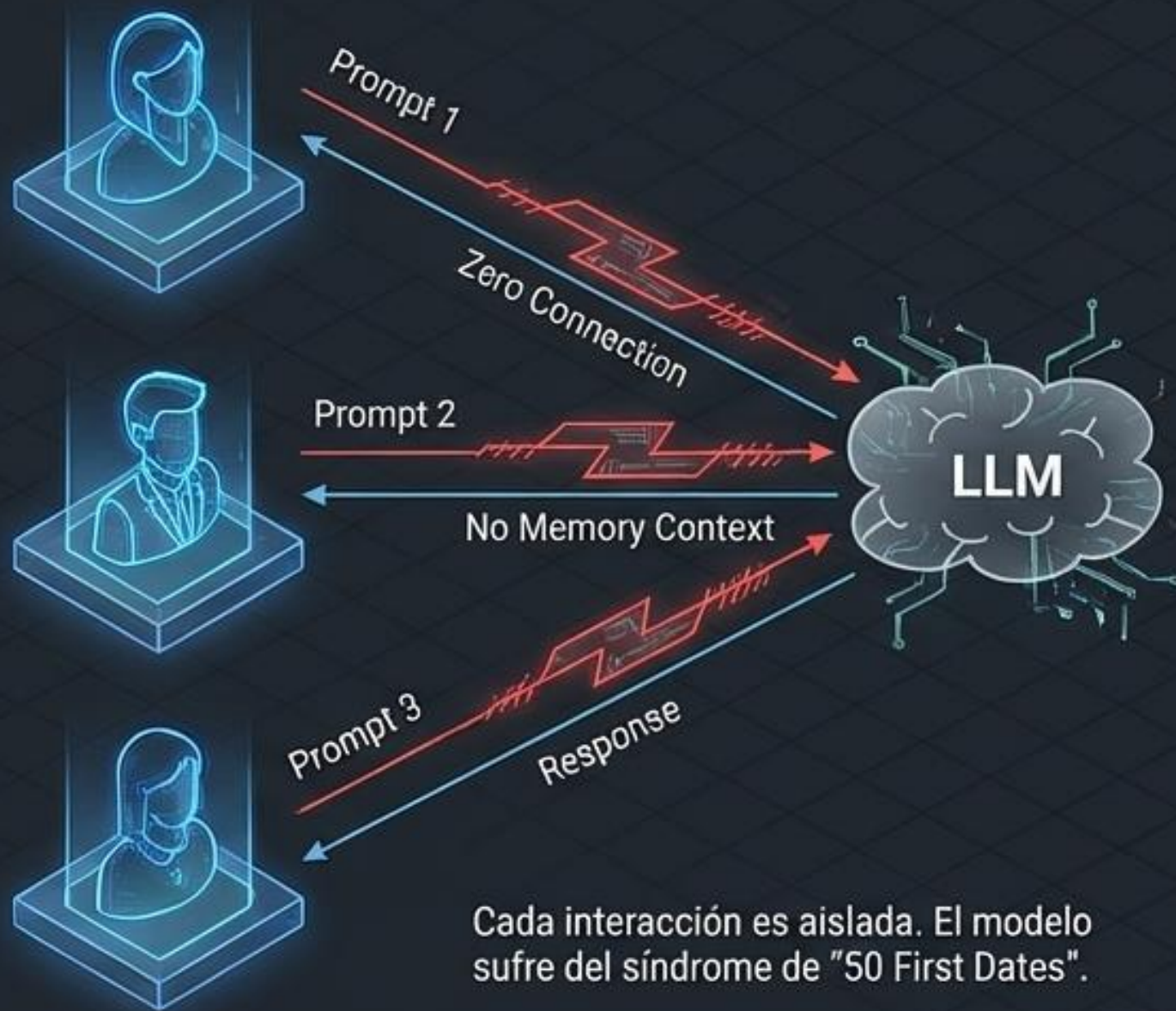
- ✓ Abstracción total del proveedor.
- ✓ Rigor en las pruebas y evaluación.
- ✓ RAG perfectamente integrado en el ciclo de vida de la aplicación.

La Anatomía de la **Inteligencia Empresarial**



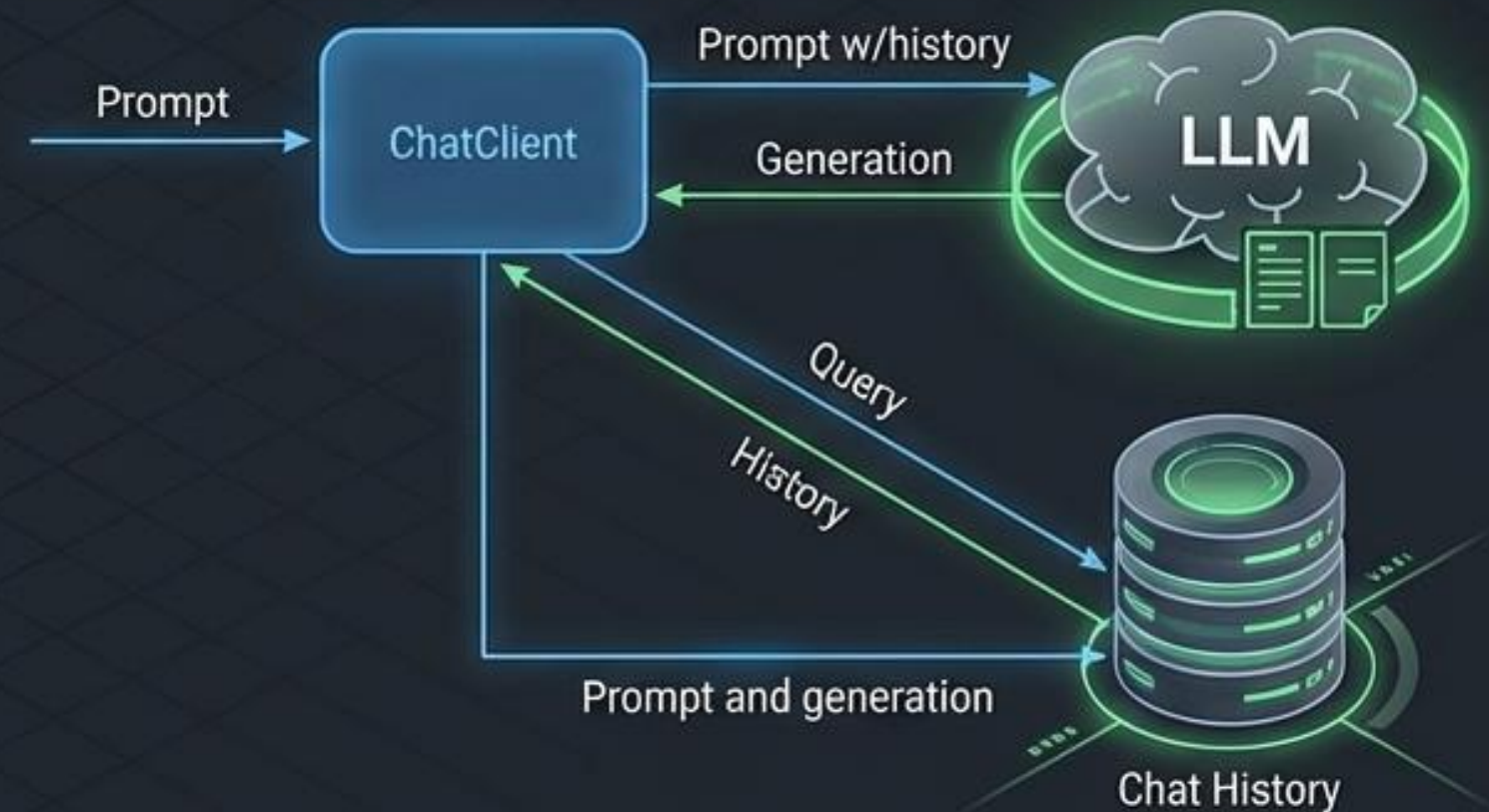
Curando la Amnesia del LLM: Memoria Conversacional

El Problema: Stateless



Cada interacción es aislada. El modelo sufre del síndrome de "50 First Dates".

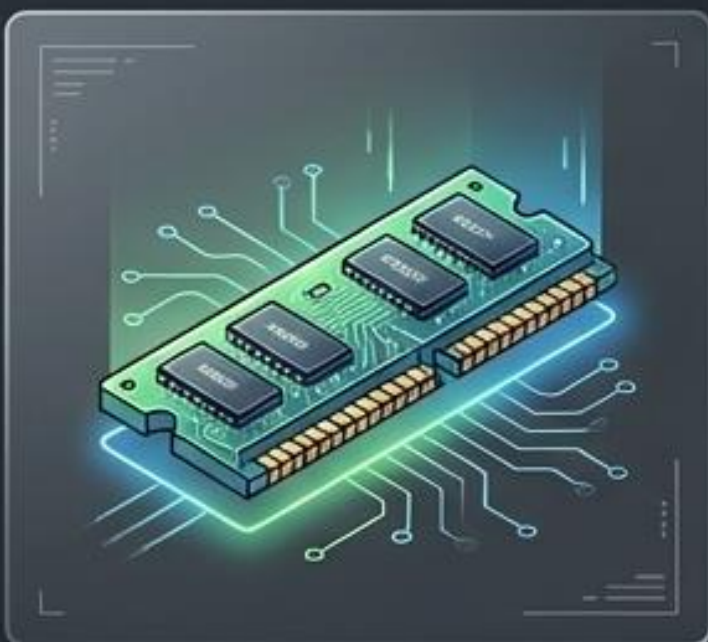
La Solución: ChatMemory Advisors



Implementaciones Clave:

1. `MessageChatMemoryAdvisor`: Inyecta historial como mensajes separados.
2. `PromptChatMemoryAdvisor`: Empaqueta historial en el System Prompt.
3. `CONVERSATION_ID`: Aísla sesiones paralelas de usuarios.

Estrategias Arquitectónicas para la Memoria a Largo Plazo



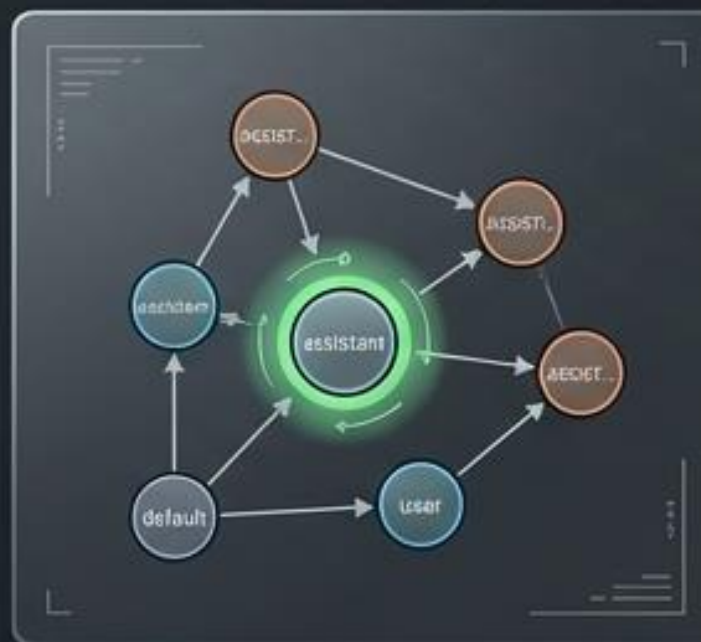
In-Memory (Map)

Volátil. Ideal para desarrollo y pruebas rápidas. No apto para producción.



Relacional / Columnar

Escalabilidad empresarial. Soporta alto volumen mediante JDBC o Cassandra. Autoconfiguración de esquemas (chat_memory).



Bases de Grafos (Neo4j)

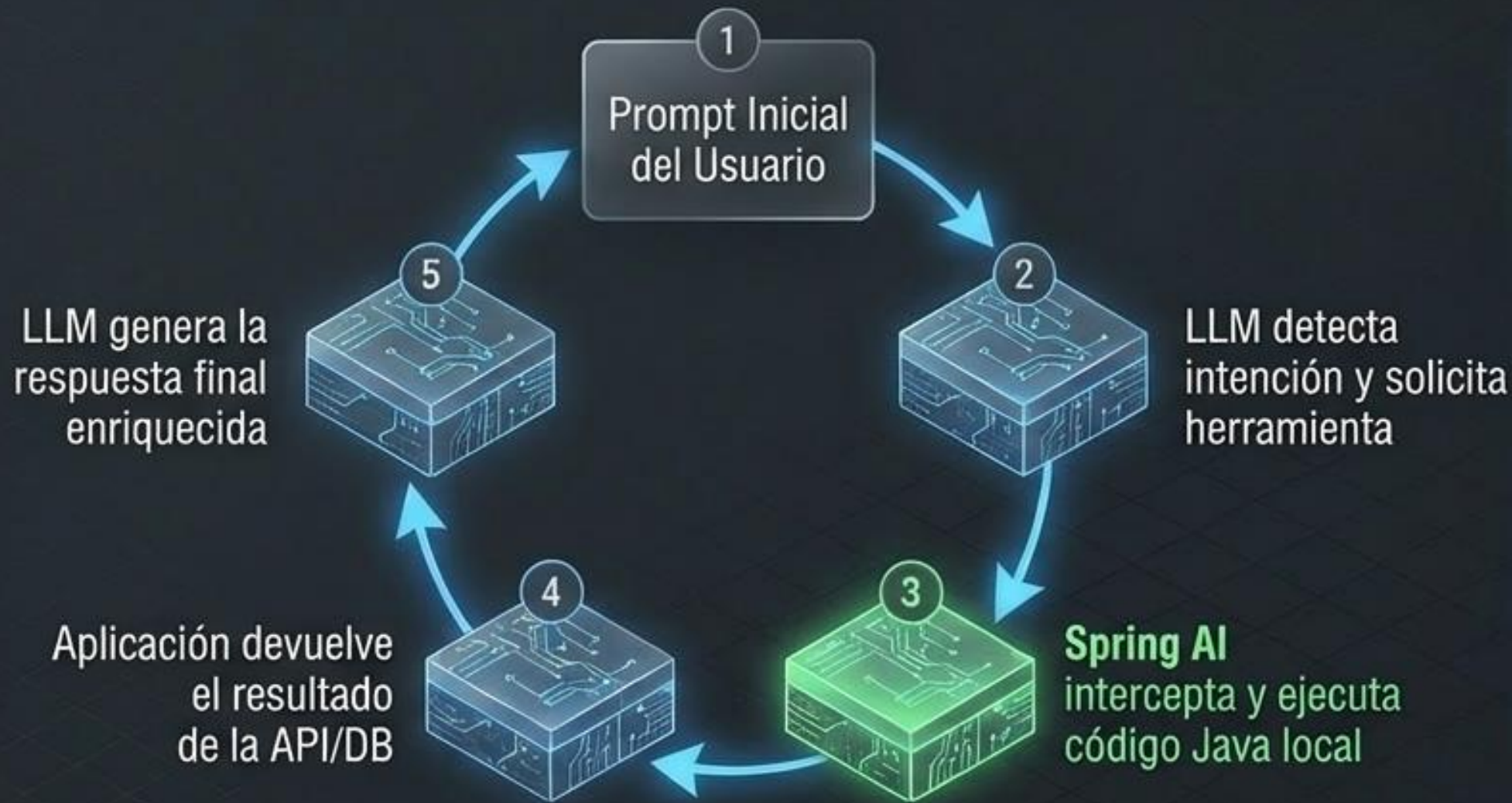
Mapea entidades y relaciones semánticas (Usuarios -> Mensajes -> Contexto) para un análisis conversacional profundo.



Vector Store

RAG aplicado a la memoria. Busca similitudes semánticas en historiales masivos en lugar de inyectar todo el texto pasado.

Dándole 'Manos' a la IA: Generación Impulsada por Herramientas



Implementación Nativa

1. Anotación `@Tool`: Expone firmas y `@ToolParam` automáticamente.
2. Interfaz `Function<T, R>`: Encapsula lógica de negocio tipada (Request/Response).

El paradigma cambia: El LLM no ejecuta el código, delega la intención a Spring AI para que se ejecute nativamente en la JVM.

Model Context Protocol (MCP): El 'USB-C' de la IA



MCP Clients (Consumidores)

Descubrimiento dinámico. Spring AI actúa como cliente para utilizar herramientas, recursos y prompts remotos sin codificar la lógica en la app principal.

MCP Servers (Proveedores)

Servidores creados con Spring Boot para exponer repositorios (`@McpResource`) y plantillas predefinidas (`@McpPrompt`).



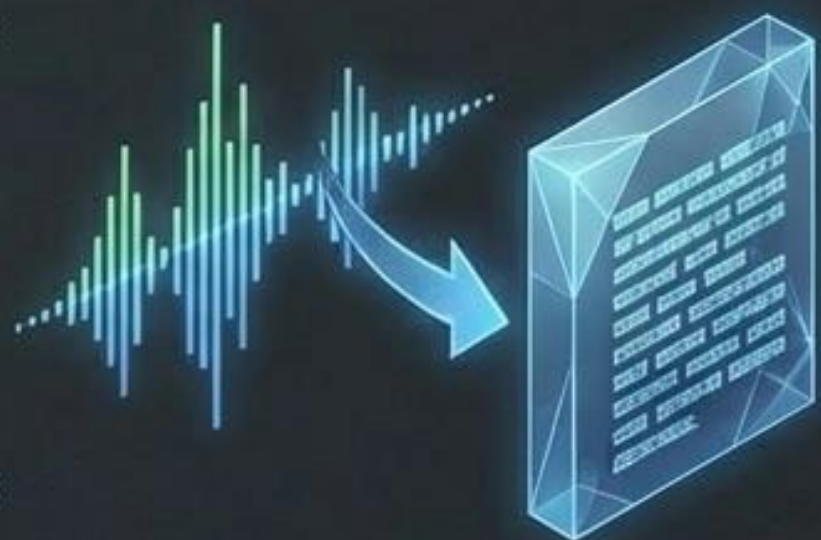
Transporte STDIO
(Procesos Locales)



Transporte HTTP+SSE
(Microservicios Web)

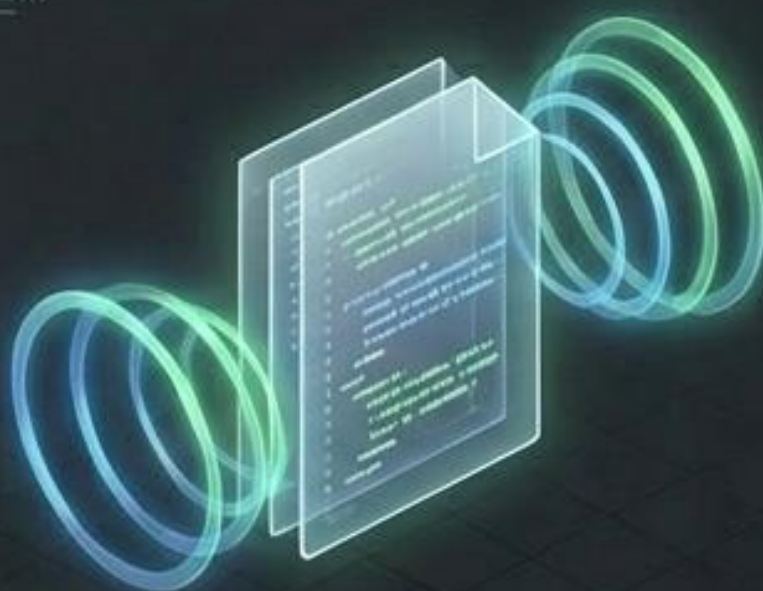
Expandiendo los Sentidos: Capacidades Multimodales

Oído: TranscriptionModel



Conversión de voz a texto nativa (Speech-to-Text). Permite ingerir archivos de audio como MP3 y procesarlos como prompts naturales.

Voz: SpeechModel



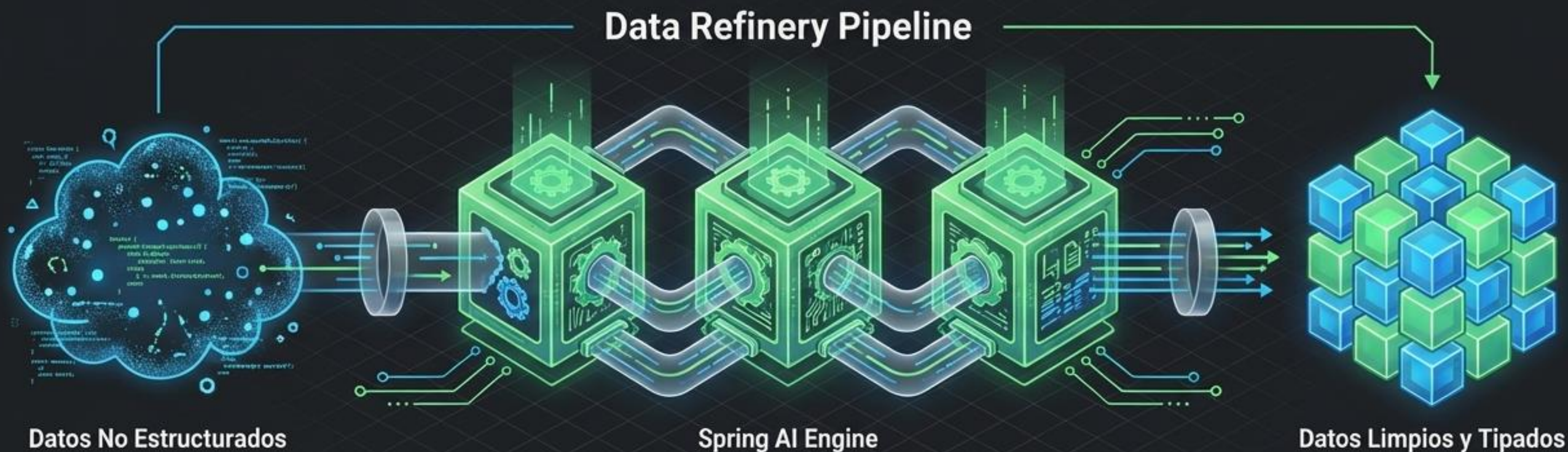
Síntesis de texto a voz (Text-to-Speech). Transforma la respuesta del agente en audio de alta calidad con perfiles acústicos configurables.

Visión Computacional



Inyección de objetos Resource directamente en el prompt. Permite al modelo analizar fotografías, leer gráficos y razonar sobre el entorno físico.

Patrones Cognitivos Reutilizables (Más Allá del Chat)



Resumen Inteligente



Condensación de documentos largos utilizando TikaDocumentReader y ChatClient para destilar información crítica.

Traducción Especializada



Enrutamiento semántico hacia idiomas meta manteniendo estrictamente el vocabulario del dominio (ej. términos legales).

Análisis de Sentimiento



Extracción estructurada de emociones. Fuerza al LLM a devolver un objeto Java fuertemente tipado: record `SentimentAnalysis(String mood, int score)`.

Seguridad Zero-Trust: Protegiendo RAG y Herramientas

Control de Acceso Dinámico en RAG

Inyección transparente de filtros de metadatos en el SearchRequest.

Si un usuario no tiene los privilegios adecuados, Spring AI modifica la consulta vectorial a nivel de base de datos para omitir los documentos clasificados.

Protección de Herramientas

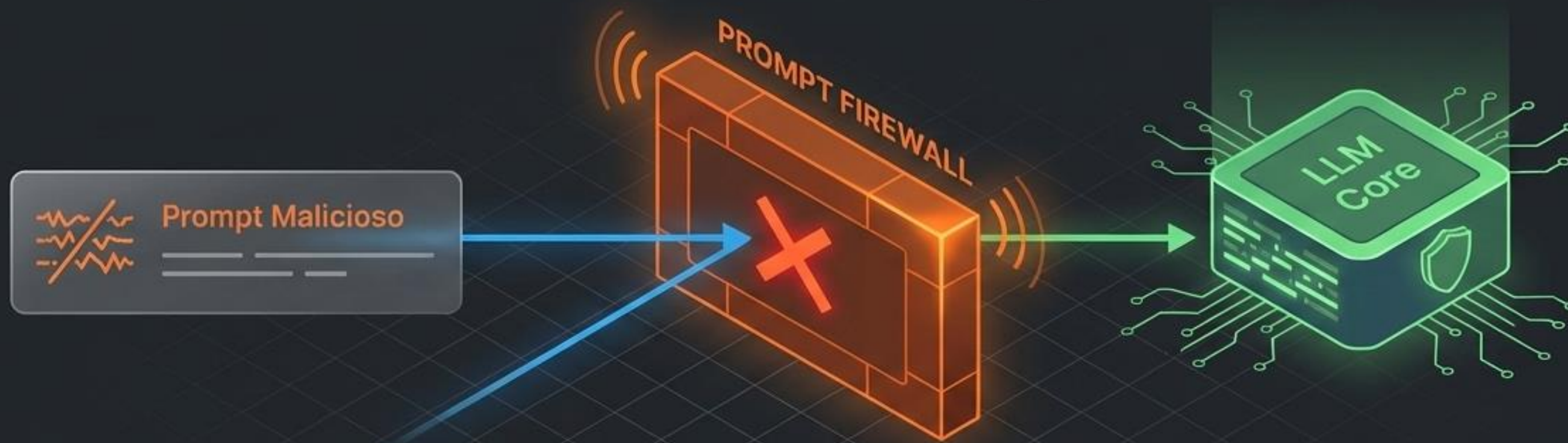
Aplicación de seguridad a nivel de método.

Uso de anotaciones estándar como `@PreAuthorize('hasRole("PREMIUM")')` sobre herramientas.

Si el LLM intenta invocar un método prohibido, la JVM lanza una excepción de denegación.



Defensas Activas contra Prompts Adversarios



SafeGuardAdvisor



Filtro en tiempo de ejecución.

Intercepta el prompt y bloquea el envío si detecta palabras clave prohibidas, violaciones de políticas o términos empresariales sensibles.

CanaryWordAdvisor



Prevención contra fugas de sistema (Prompt Leaking).

Injecta un token aleatorio en las instrucciones; si el LLM revela ese token en la respuesta, significa que fue manipulado y la respuesta se anula.

ModerationModel



Delegación a APIs externas dedicadas.

Clasifica y modera la entrada y salida para detectar toxicidad, violencia o inyecciones de prompt antes de que lleguen al usuario.

Observabilidad: Midiendo el Latido de la IA



Trazabilidad Distribuida: Radiografía de una Petición



Potenciado por **OpenTelemetry: Identificación instantánea de cuellos de botella.**
Descubre con precisión milimétrica si la latencia proviene de la red local, de una búsqueda ineficiente en el Vector Store, o del proveedor del LLM.

Orquestación: De Tareas Aisladas a Flujos de Trabajo



Encadenamiento (Chaining)

Flujo secuencial. La salida de un nodo alimenta directamente al siguiente (Ej. Extraer texto -> Traducir -> Resumir).



Enrutamiento (Routing)

Bifurcación condicional. El modelo clasifica la intención de la entrada y desvía la petición a herramientas o sub-agentes específicos según la necesidad.



Paralelización (Parallelization)

Ejecución simultánea. Tareas independientes operan al mismo tiempo y un nodo final consolida los resultados, reduciendo drásticamente la latencia global.

La Frontera Autónoma: Planificación Agentica con Embabel

Del Flujo Imperativo...

Tradicionalmente, los desarrolladores codifican lógicas paso a paso rígidas ("Haz A, luego evalúa B, luego ejecuta C").

@Action

...A la IA Declarativa

Definimos Herramientas y Metas. El motor agéntico evalúa la intención del usuario y crea dinámicamente la ruta de ejecución óptima.

@AchievesGoal

Impacto Arquitectónico: La aplicación se adapta de manera autónoma a requerimientos imprevistos, orquestando herramientas basadas en Spring AI y MCP para resolver problemas sin intervención manual.

El Stack Definitivo de IA Empresarial



La IA trasciende los scripts.

Spring AI fusiona los modelos LLM modernos con los estándares de seguridad, escalabilidad y observabilidad de la industria.

El paradigma ha cambiado.

La lógica evoluciona, pero el ecosistema sigue siendo Java.

Hemos construido el puente definitivo hacia la empresa inteligente.